

NLP Mini Project 1 Notebook Explanation Guide

Text Generation with 4-gram Language Models

Notebook: C:\Users\MSI\Documents\nlp_mini_project_1\NLP_Mini_Project_1_Text_Generation.ipynb

Corpus: Khmer Wikipedia article about ប្រាសាទ អង្គរវត្ត

Purpose: This guide explains what each notebook section/cell does and how to interpret the results.

1. Project Overview

The notebook completes the NLP assignment using one corpus from Khmer Wikipedia. It builds two 4-gram language models, evaluates them with perplexity, and demonstrates text generation.

- LM1 is an unsmoothed backoff 4-gram language model.
- LM2 is an interpolated 4-gram language model using add-k smoothing.
- The corpus is split into 70% training, 10% validation, and 20% testing.
- Vocabulary is limited, and rare/out-of-vocabulary tokens are replaced with <UNK> as required.

2. Cell-by-Cell Explanation

Notebook Section	What the Code Does	Why It Matters
Title and Assignment Tasks	Introduces the project, corpus, and required assignment tasks.	Shows the notebook is aligned with the project instruction.
Import Libraries and Set URL	Imports math, random, re, pandas, Counter/defaultdict, requests, and BeautifulSoup. Stores the Angkor Wat Wikipedia URL.	Prepares tools for scraping, preprocessing, counting n-grams, evaluation, and result tables.
Load Corpus	Defines FALLBACK_CORPUS and fetch_wikipedia_text(). The function tries to scrape the Wikipedia page and falls back to stored text if internet is unavailable.	Makes the notebook self-contained and reliable for presentation/grading.
Preprocess, Tokenize, and Split	Removes bracket references, separates punctuation, normalizes spaces, tokenizes with split(), then splits tokens into train/validation/test.	Creates clean token sequences for language modeling while following the assignment tokenizer option.
Vocabulary Limiting	Builds a vocabulary of the most frequent training tokens and replaces other tokens with <UNK>.	Directly satisfies the assignment requirement to limit vocabulary and use <UNK>.
N-gram Counting	Counts unigrams, bigrams, trigrams, and 4-grams, plus context counts and possible next-token followers.	Provides the statistics needed for probability calculation and generation.
LM1 Backoff Model	Tries 4-gram probability first; if unseen, backs off to 3-gram, 2-gram, then unigram.	Handles unseen higher-order contexts without smoothing.
LM2 Interpolation Model	Combines unigram, bigram, trigram, and 4-gram probabilities with lambda weights and add-k smoothing.	Tests a smoother model that can assign probability to unseen n-grams.
Perplexity and Generate Functions	Computes perplexity and generates text token-by-token from a seed phrase.	Supports the evaluation and text generator requirements.
Train and Tune Models	Trains LM1 and searches several lambda/k combinations for LM2 using validation perplexity.	Uses validation data to choose hyperparameters as required.

Test Evaluation Results	Evaluates both models on the test set and displays a comparison table.	Shows which model predicts unseen test text better.
Vocabulary Size Experiment	Compares vocabulary caps of 50, 100, and 150.	Adds deeper analysis of how vocabulary size affects <UNK> and perplexity.
Text Generation Examples	Generates text from several seed phrases: អង្គរវត្ត, ប្រាសាទ, and ស្ថាបត្យកម្ម.	Demonstrates the text generator requirement with multiple examples.
Interactive Demo UI	Provides a text box, model selector, length slider, and generate button.	Makes the project easier to present live.
Conclusion	Summarizes model comparison, limitations, and future improvements.	Shows understanding beyond simply running code.

3. Main Result Explanation

Metric	Value	Meaning
Corpus characters	8,711	Amount of raw article text loaded from the Angkor Wat article.
Total tokens	572	Number of tokens after preprocessing and split-based tokenization.
Training tokens	400	70% of the corpus used to learn n-gram counts.
Validation tokens	57	10% used to choose lambda and k for LM2.
Testing tokens	115	20% used for final model evaluation.
Vocabulary size	100	Maximum vocabulary size used in the main experiment.
Validation <UNK>	37	Number of validation tokens outside the training vocabulary.
Test <UNK>	91	Number of test tokens outside the training vocabulary.

The high number of <UNK> tokens shows that many words in validation/testing were not among the most frequent training tokens. This is expected because the corpus is small and the vocabulary is limited as required by the assignment.

4. Model Comparison Result

Model	Method	Best Parameters	Validation Perplexity	Test Perplexity
LM1	Unsmoothed backoff 4-gram	No lambda/k	-	2.6255
LM2	Interpolation + add-k smoothing	lambdas=(0.4, 0.3, 0.2, 0.1), k=0.01	16.3922	3.7461

Interpretation: lower perplexity means better next-token prediction. LM1 has lower test perplexity than LM2, so LM1 performs better in this experiment.

Reason: the corpus is small, so LM1 can benefit from exact local patterns. LM2 is smoother and more flexible, but it spreads probability across more vocabulary items, which can reduce accuracy on this small test set.

5. Vocabulary Size Experiment

Vocabulary Cap	Actual Vocab	Validation <UNK>	Test <UNK>	LM1 Test PPL	LM2 Test PP	Best k
50	50	38	91	2.4669	3.3655	0.01
100	100	37	91	2.6255	3.7461	0.01
150	150	37	91	2.7391	4.2633	0.01

Interpretation: a smaller vocabulary creates more <UNK> tokens, while a larger vocabulary keeps more words but can make n-gram counts more sparse. In this experiment, vocabulary cap 50 gives the lowest perplexity, but cap 100 is a reasonable main choice because it preserves more real vocabulary while still keeping the model simple.

6. Text Generation Result Explanation

The generator starts with a seed phrase and repeatedly predicts the next token using the selected language model.

Seed	LM1 Behavior	LM2 Behavior
អង្គរវត្ត	Shorter and more pattern-based; often follows phrases seen in the corpus.	More varied, but can include more <UNK> and less readable transitions.
ប្រាសាទ	Produces a clearer sentence about Angkor Wat as a temple.	Produces a longer mixed sentence with historical and source terms.
ស្ថាបត្យកម្ម	Uses corpus phrases related to architecture when context is found.	Can be more random because interpolation samples from more possible tokens.

For live demonstration, use LM1 Backoff with length 20-30. Longer output is possible, but it becomes less readable because this is a simple n-gram model trained on a small corpus.

7. Why <UNK> Appears

<UNK> appears because the assignment requires limiting vocabulary size and replacing the remaining tokens with <UNK>. It is not an error. It shows that rare or unseen tokens are being handled according to the project instruction.

Limit the vocabulary size and replace the rest of the tokens as <UNK>.

8. Limitations and Future Improvement

- Khmer tokenization with split() is simple but not ideal because Khmer word boundaries are not always separated by spaces.
- The corpus is small, so generated text cannot be as natural as modern neural language models.
- Some English words and source labels remain because they are present in the Wikipedia article.
- Future improvement: use a Khmer word segmentation tool and a larger Khmer corpus.

9. Short Presentation Script

This notebook builds two 4-gram language models using the Khmer Wikipedia article about Angkor Wat. I cleaned and tokenized the text, split it into training, validation, and testing sets, limited the vocabulary, and replaced rare tokens with <UNK>. LM1 uses backoff, while LM2 uses interpolation with add-k smoothing. I used the validation set to choose lambda and k for LM2, then evaluated both models on the test set using perplexity. LM1 achieved lower test perplexity, so it performed better in this experiment. The generator demo shows how the trained model predicts new text from a seed phrase.